Efficient Hardware Architectures for 2-D BCH Codes in the Frequency Domain for Two-Dimensional Data Storage Applications

Arijit Mondal and Shayan Srinivasa Garani

Department of Electronic Systems Engineering, Indian Institute of Science, Bengaluru 560012, India

We propose fast and efficient hardware architectures for a 2-D Bose–Chaudhuri–Hocquenghem (BCH) code of size $n \times n$, with a quasi-cyclic burst error correction capability of $t \times t$, in the frequency domain for data storage applications. A fully parallel encoder with the ability to produce an output every clock cycle was designed. Using conjugate class properties of finite fields, the algorithmic complexity of the encoder was significantly reduced, leading to a reduction in the number of gates by about 94% of the brute-force implementation per 2-D inverse discrete finite field Fourier transform (IDFFFT) point for a 15 × 15, $t = 2$ 2-D BCH code. We also designed a pipelined, low-latency decoder for the above encoder. The algorithmic complexity of various pipeline stages of the decoder was reduced significantly using finite field properties, reducing the space complexity of the entire decoder. For a particular case of $n = 15$ and $t = 2$, the architectures were implemented on a Kintex 7 KC-705 field-programmable gate array (FPGA) kit, giving high throughputs of 22.5 and 5.6 Gb/s at 100 MHz for the encoder and decoder, respectively.

***Index Terms*— 2-D Bose–Chaudhuri–Hocquenghem (BCH) codes, frequency-domain encoders/decoders, magnetic disks, two-dimensional magnetic recording (TDMR).**

I. Introduction

ERROR correction codes (ECCs) are essential to transmission and data storage systems to protect the information from errors introduced by noisy communication channels. There are two main classes of ECCs, namely algebraic and iterative ECCs. While iterative ECCs such as low-density parity-check (LDPC) codes provide improved performance in the waterfall region, algebraic ECCs, such as Bose–Chaudhuri–Hocquenghem (BCH) and Reed–Solomon (RS) codes, provide guaranteed error correction capability irrespective of the waterfall or error floor regions [1]. In principle, both iterative and algebraic ECCs are practically useful depending on the constraints of the storage channel. Due to recent advancements in higher dimensional data storage technologies such as shingled [2] and two-dimensional magnetic recording (TDMR) [3], 3-D NAND flash memories [4], and holographic memories [5], native 2-D signal processing and coding techniques are required to overcome intersymbol interference (ISI) and noise leading to 2-D burst and random errors [6]–[8].

Imai [9] introduced the theory of 2-D cyclic ECCs. Blahut [10] introduced higher dimensional finite field transforms for realizing algebraic ECCs. Analogous to Imai's technique [9], encoding and decoding algorithms for 2-D ECCs were studied in the frequency domain in [11]. It must be noted that time-domain encoding of 2-D ECCs requires computationally intensive operations working with large-sized generator and parity-check tensors. A frequency-domain approach [10], on the other hand, does not require such operations and provides rate flexibility. However, the frequency-domain approach involves computation of the inverse discrete finite field Fourier transform (IDFFFT). A reduced complexity algorithm for the evaluation of the IDFFFT thus becomes critical to the design of hardware architectures for an encoder based on the frequency-domain approach.

One way to realize BCH codes in 2-D is through a product of 1-D BCH codes, which was first introduced by Elias [12] and further studied in the spectral domain by Blahut [10]. Efficient encoding and decoding architectures for a product of two (n, k), t-error correcting BCH codes were proposed in [13], where a low-complexity 2-D encoder was designed, and parallel instances of 1-D BCH decoders [14] were used for decoding. Though such codes are able to correct $n \times t$ bursts along a row or column, product codes do not provide good coding rates applicable to high-density storage channels. Consequently, correction of square bursts of large sizes is not possible. This motivates us toward the design of 2-D BCH encoders and decoders, which provides better rates and the ability to correct large square burst errors.

Decoding algorithms for a 2-D BCH code were introduced by Madhusudhana and Siddiqi [15], who proposed the IBA-I algorithm by extending Blahut's BA-I [10] algorithm to two dimensions. A 2-D decoding architecture is challenging, due to the large computational complexity of the 2-D DFFFT and IDFFFT algorithms. A recursive extension (RE) over the $2t \times 2t$ syndrome array corresponding to the spectral nulls of the code needs to be performed along the rows and columns to produce the complete syndrome array of size $n \times n$ for all spectral positions of the code. Madhusudhana and Siddiqi [15] suggested a serial implementation of the DFFFT, Berlekamp–Massey (BM) engine, RE, and IDFFFT units. Such serial implementation would require a large number of clock cycles, leading to large latencies, making it inefficient for high-speed data storage channels.

In [16], 1-D BCH decoders were designed using ROMs as lookup tables for cases when $t \leq 4$. This technique is useful for specific, smaller error correction requirements.

Manuscript received December 13, 2020; accepted February 8, 2021. Date of publication February 22, 2021; date of current version April 19, 2021. Corresponding author: S. Srinivasa Garani (e-mail: shayangs@iisc.ac.in).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TMAG.2021.3060807.

Digital Object Identifier 10.1109/TMAG.2021.3060807

0018-9464 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

While there are many efficient architectures for realizing 1-D ECCs, hardware architectures for 2-D ECCs have hardly been explored. In this article, we are interested in building efficient hardware architectures for 2-D BCH codes capable of correcting quasi-cyclic bursts of size $t \times t$ for applications in shingled magnetic recording, TDMR, and 3-D NAND flash memories. We propose high-throughput and low-latency design architectures for 2-D BCH codes for correcting quasi-cyclic bursts of size $t \times t$ based on the improvements to the IBA-I algorithm proposed in [15] attuned to the hardware requirements.

The novelty in our encoder design lies in reducing the complexity of the IDFFFT unit, using some algebraic properties of finite fields. We designed a fully parallel encoder that produces a 2-D BCH codeword every clock cycle. We also designed a novel decoder architecture by reducing the algorithmic complexities of various stages in the IBA-I algorithm. First, we used conjugate class properties to reduce the space complexities of the DFFFT and IDFFFT units, thus making parallel implementation feasible. Second, unlike IBA-I [15] where the BM engine computations on rows (columns) need to be done serially to compute the common connection polynomial, we process the BM engines in parallel and subsequently compute the common connection polynomial from the parallel BM engine outputs. This allows high-throughput and low-latency design architectures amenable to TDMR. Using our algorithmic reduction, we designed a pipelined decoder having a latency of $(2t+5)$, which can correct an $n \times n$ 2-D BCH code every $2t$ clock cycles. To the best of our knowledge, this is the first time such a design architecture is proposed toward a low-latency, high-throughput encoder and decoder for 2-D BCH codes applicable to multidimensional data storage channels.

This article is organized as follows. In Section II, we describe the 2-D BCH code construction. This is followed by a reduced complexity encoding algorithm followed by the hardware architecture in Section III. In Section IV, we discuss the modified decoding procedure amenable to parallel implementation along with the hardware architecture. In Section V, we discuss the field-programmable gate array (FPGA) implementation results and comparisons with other relevant works, followed by conclusions in Section VI.

II. Theoretical Background

In this section, we will discuss the mathematical preliminaries pertaining to 2-D cyclic codes and spectral-domain encoding and decoding, useful toward the rest of this article.

A. 2-D Finite Field Fourier Transforms

Let $[c_{i,i'}]$ denote an $n \times n'$ array over GF(q) and suppose that n and n' divide $q^m - 1$ for some integer m. Let α and β denote elements of GF(q^m) of orders n and n', respectively. Then, the 2-D DFFFT array $[C_{j,j'}]$ of size $n \times n'$ is given by (1a) [10]. Similarly, the 2-D IDFFFT array $[c_{i,i'}]$ is given by (1b) [10]

$$C_{j,j'} = \sum_{i=0}^{n-1} \sum_{i'=0}^{n'-1} \alpha^{ij} \beta^{i'j'} c_{i,i'} \tag{1a}$$

$$c_{i,i'} = \frac{1}{\text{mod}(n,q)} \frac{1}{\text{mod}(n',q)} \sum_{j=0}^{n-1} \sum_{j'=0}^{n'-1} \alpha^{-ij} \beta^{-i'j'} C_{j,j'}. \tag{1b}$$

Let $[c_{i,i'}]$ be the 2-D time domain array with (i, i') denoted as time-domain variables. $[C_{j,j'}]$ is called the spectrum of $[c_{i,i'}]$ over the frequency variables (j, j'). Equations (1a) and (1b) represent the discrete finite field Fourier transform pair.

Here, one has to deal with GF(q^m) where $q^m = \text{lcm}(n, n') + 1$ [10], where lcm$(\cdot)$ is the least common multiple. If $n = n'$, we have $q^m = n + 1$, leading to a smaller size of the extension field amenable to hardware design.

In the 2-D spectrum, only a set of frequencies may be specified, which yields a binary codeword when 2-D IDFFFT is performed [10]. This lays a set of rules on the spectral positions known as conjugacy constraints, as shown in the following equation:

$$C^2_{j,j'} = C_{(2j \bmod n),(2j' \bmod n)}. \tag{2}$$

Using the conjugacy constraints in (2), the frequency positions in the spectrum are segregated into different conjugate classes. Only one member of a conjugate class can be used to encode information, whereas the other members are obtained through conjugacy constraints.

B. Cyclic and Quasi-Cyclic Bursts for a 2-D Code

For an error word array $[e]$ of size $n \times n$ in GF(q), Imai [9] defined a burst error as the size of minimum sub-matrix $[b]$ in $[e]$, which contains non-zero components of error $[e]$. The starting position of the burst error pattern is referred to as the upper left corner of $[b]$ in $[e]$. Taking the position of this pattern as (i, i'), the error $[e]$ can be represented as

$$e(x, y) = (x^i y^{i'} b(x, y)) \,\text{modulo}(x^n - 1, y^n - 1) \tag{3}$$

where $e(x, y)$ and $b(x, y)$ are bi-variate polynomial representations of $[e]$ and $[b]$, respectively. If $0 \le i \le n - b_1$ and $0 \le i' \le n - b_2$, then $[e]$ is known as a $b_1 \times b_2$ 2-D burst error. If $0 \le i \le n - 1$ and $0 \le i' \le n - 1$, then $[e]$ is known as a $b_1 \times b_2$ 2-D cyclic burst error. A cyclic burst error can wrap around in two dimensions because of the modulo polynomial operations. A quasi-cyclic burst $[e_{i,i'}]$ of area $t_1 \times t_2$ may have non-zero values only at positions $i \in \{i_1, i_2, \ldots, i_{t_1}\}$ and $i' \in \{i_1, i_2, \ldots, i_{t_2}\}$. At other positions, the value must be zero. It is evident that the cyclic burst is a special type of quasi-cyclic burst. Thus, any decoding algorithm, which can correct quasi-cyclic bursts of size $t_1 \times t_2$, will be able to correct cyclic bursts of size $t_1 \times t_2$.

C. 2-D BCH Codes

A 2-D BCH code [15] is defined as the set of arrays $[c_{i,i'}]$ whose 2-D DFFFT (for $n = n'$) is given by

$$C_{a(m(j)+j'),a'(m(j)+j')+1} = 0 \tag{4}$$

for $1 \le j \le 2t$ and $1 \le j' \le 2t$. It can be shown that the minimum distance of the code is at least $2t + 1$ given $\gcd(a, n) = 1$. If we choose $a = 1$, $a' = 0$ and $m(k) = 0$ for $1 \le k \le 2t$, the zeros of the spectrum are reduced to a convenient structure amenable for architectural designs.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

These 2-D BCH codes with the modified zero structure are represented by the following equation:

$$C_{j,j'} = 0; \quad \text{for } 1 \le j \le 2t,\ 1 \le j' \le 2t. \tag{5}$$

It has been proved in [15] that a 2-D BCH code with zeros $C_{j,j'} = 0$, for $1 \le j \le 2t$ and $1 \le j' \le 2t$, has a quasi-cyclic burst error correction capability $t \times t$. It can correct every 2-D quasi-cyclic burst with area at most $t \times t$ and does not uniquely decode quasi-cyclic bursts with area greater than $t \times t$. Thus, it has a unique syndrome for each 2-D quasi-cyclic burst of area at most $t \times t$. As a cyclic burst is a special type of quasi-cyclic burst, a 2-D BCH code as defined in (5) can correct any cyclic burst with area at most $t \times t$.

Madhusudhana and Siddiqi [15] proposed improved Blahut's decoding algorithm-I (IBA-I) for correction of cyclic and quasi-cyclic burst errors in a 2-D BCH code. Using some finite field properties, we reduce the complexity of IBA-I algorithm. This leads to a significant reduction in the hardware requirements for the DFFFT and IDFFFT units. Also, the modified decoding algorithm allows parallel implementations of the BM engine. We shall discuss the algorithm and architectures for the 2-D BCH encoder and decoder in detail in Sections III and IV. Throughout this article, we deal with square arrays of size $n \times n$.

D. Tiling of 2-D BCH Codes for Applications in 2-D Magnetic Storage and 3-D Storage Applications

We propose the following tiling mechanism for a 2-D BCH codes of size $n \times n$ with quasi-cyclic burst error correction capability $t \times t$. Consider a magnetic array of dimensions $d_1 \times d_2$. The bottom-left corner of the $d_1 \times d_2$ array is indexed $(0, 0)$. Arrays of size $n \times n$ encoded by a 2-D BCH encoder are tiled in such a way that every square formed by the points $\{(i, j), (i+n, j), (i, j+n), (i+n, j+n)\}$ is a 2-D BCH code if $\text{mod}(i, n) = \text{mod}(j, n) = 0$ and $0 \le i \le d_1$, $0 \le j \le d_2$. Thus, every $n \times n$ square in Fig. 1 is a 2-D BCH coded array; we call this a cell. We observe the following (shown in Fig. 1).

1) If at most a single burst of size $t \times t$ (denoted by the smaller maroon squares in Fig. 1) has occurred in every cell, a 2-D BCH decoder with a burst error correction capability of $t \times t$ will be able to correct all the bursts in the array.
2) A $t \times t$ burst that overlaps 2 or more cells can also be corrected, as long as all those cells which the burst encompasses are not affected by any other burst.
3) A burst larger than $t \times t$ can also be corrected if it is broken into smaller bursts of size lesser than or equal to $t \times t$.

The choice of the size of the code attuned to the $t \times t$ burst error capability can be modeled using the proposed approach in [17]. Under the following conditions, we can provide guaranteed multiple burst error correction capability.

1) No burst of size greater than $t \times t$ are introduced by the channel.
2) Let the starting position of a $t \times t$ burst be denoted by (i, j). No other burst occurs within the area $i - n \le x \le i + n + t$, $j - n \le y \le j + n + t$. This must be true for all $t \times t$ bursts occurring in the area $d_1 \times d_2$ to guarantee error correction.

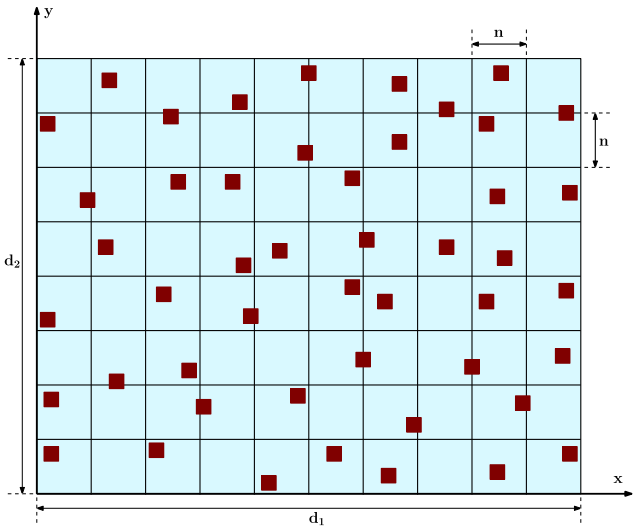


Fig. 1. Tiling of an $n \times n$ 2-D BCH code with a burst error correction capability $t \times t$. The $n \times n$ squares represent the individual 2-D BCH encoded arrays and the small maroon squares represent a burst of size $t \times t$.

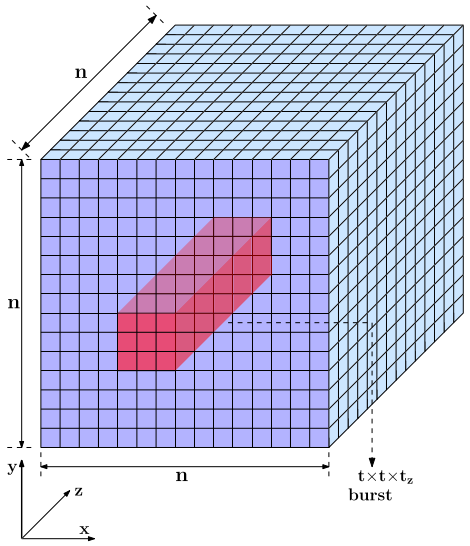


Fig. 2. Tiling of n 2-D BCH codes of dimensions $n \times n$ in 3-D space for correction of burst errors along the z-axis. A $t \times t \times t_z$ burst is shown in red.

The 2-D BCH code can also be used for 3-D storage devices, such as 3-D NAND flash memories. Analogous to stacking 1-D codes in a 2-D array, 2-D codes can be stacked in a 3-D space. The advantage of such stacking is the capability to correct bursts along the z-axis. For example, if we stack n number of $n \times n$ 2-D BCH codes in 3-D, we will get an $n \times n \times n$ 3-D array. If each $n \times n$ 2-D BCH code is capable of correcting a $t \times t$ burst, the overall arrangement will be able to correct a $t \times t \times n$ 3-D burst. Such a tiling is shown in Fig. 2 where a $t \times t \times t_z$ burst is shown in red.

III. Encoding Procedure and Hardware Architecture

In this section, we discuss the construction of a 2-D BCH code and the encoding algorithm. We also discuss how we

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Encoding Algorithm for a 2-D BCH Code

Input: Message vector M.
Output: Encoded $n \times n$ array $[c_{i,i'}]$.
Outline of steps:
Step 1: Let the spectrum of a 2-D BCH code of size $n \times n$ be denoted by $[C_{j,j'}]$. Zeros are encoded into spectral positions corresponding to $1 \leq j \leq 2t$ and $1 \leq j' \leq 2t$. The conjugate positions corresponding to the $2t \times 2t$ zero positions are also nulled. The remaining positions in the $n \times n$ spectrum are used to encode frequencies corresponding to the message vector.
Step 2: The message vector of length k denoted by $M = [c_1\, c_2\, c_3 \ldots c_l]$ is divided into sub-messages $\{c_j\}_{j=1}^{l}$, where l denotes the number of conjugate classes used for encoding messages. $\{c_j\}_{j=1}^{l}$ corresponds to elements in GF(2^m).
Step 3: The sub-messages $\{c_j\}_{j=1}^{l}$ are encoded into the respective non-zero spectral positions, giving the complete spectrum $[C_{j,j'}]$.
Step 4: 2-D IDFFFT of $[C_{j,j'}]$ is performed to obtain the encoded codeword in time domain i.e., $[c_{i,i'}]$.

reduce the complexity of the encoder, followed by the encoder design architecture.

A. 2-D BCH Code Construction and Encoding Algorithm

We consider a binary $n \times n$ 2-D BCH code given by (5). The corresponding conjugate positions of the frequencies in the $2t \times 2t$ region of the spectrum also become zeros. Information is encoded into the remaining spectral positions. Subsequently, we use 2-D IDFFFT to obtain the binary codeword from the frequency domain.[1] In the frequency domain, we have an extension field GF(2^m), where m is chosen such that $2^m = n + 1$.

The central ideas for frequency-domain encoding were originally given by Blahut [10]. However, in [10], the exact details of splitting the message vector were not outlined. These details were mentioned in our earlier work dealing with product codes [13]. However, the spectrum of 2-D BCH codes is different. We mention those details in Algorithm 1 for completeness.

It may be noted that in step 2, a certain conjugate class may contain lesser elements than m. In such a case, we do the following to map it to a m-bit vector, as illustrated by the following example. For GF(2^4), with primitive polynomial $p(x) = x^4 + x + 1$, a conjugate class of size 2 contains $\alpha^5 = 0110$ and $\alpha^{10} = 1110$. It may be observed that the last two bits are the same in α^5 and α^{10}. Thus, the different submessages of 2 bits can be transformed as $00 \rightarrow 0000(0)$, $01 \rightarrow 0110(\alpha^5)$, $10 \rightarrow 1000(1)$ and $11 \rightarrow 1110(\alpha^{10})$, by padding additional bits.

Steps 1–3 are illustrated for a 2-D BCH code of size 15×15 with $t = 2$ in Figs. 3 and 4. The spectrum is over GF(2^4), using the primitive polynomial $p(x) = x^4 + x + 1$. Using the conjugacy constraints in (2), the frequency positions are divided into 59 conjugate classes according to (2), as shown in Fig. 3. The 44 spectral nulls represented in yellow in Fig. 3 are spread among 11 conjugate classes represented by C-a, C-a, ..., C-k. The remaining 181 frequency positions used for encoding the message are spread among 48 conjugate classes represented by C-1, C-2, ..., C-48 in Fig. 3. A random binary message vector of length 181 was divided into sub-messages, as shown in Fig. 4. Each sub-message is mapped into elements of GF(2^4) and subsequently encoded into the respective conjugate class position marked in green in Fig. 3. This point is denoted as the insertion point (IP) for a particular conjugate class. The rest of the positions for a particular conjugate class are filled with the conjugates of the corresponding sub-message, following (2). This results in the complete encoded spectrum for the message vector, as shown in Fig. 4.

The 2-D BCH code offers better rates than BCH product codes for the same code dimensions [13]. The example 2-D BCH code described above offers different choices for rates, as shown in Table I. For $t = 2$, we observe a rate of (181/225) for the 2-D BCH code, whereas in case of a product of two (15, 7), $t = 2$ error correcting BCH codes, which has the same dimension 15×15, we observe a poorer rate of (49/225).

[1]The terms spectral domain and frequency domain are used interchangeably to mean the same.

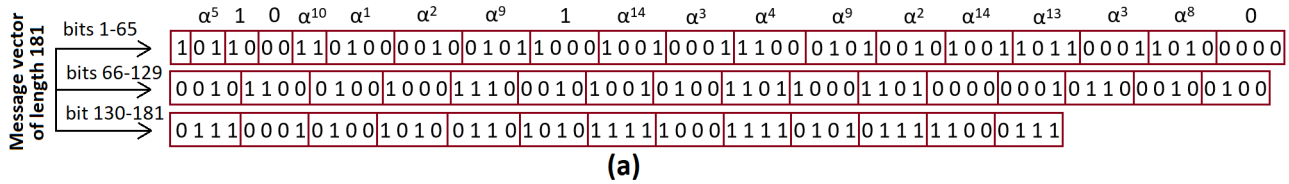
j,j'	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0
1	C-a	C-b	C-c	C-d	C-10	C-11	C-12	C-e	C-f	C-13	C-14	C-k	C-15	C-16	C-9
2	C-e	C-a	C-f	C-b	C-13	C-c	C-14	C-d	C-k	C-10	C-15	C-11	C-16	C-12	C-9
3	C-g	C-h	C-i	C-j	C-18	C-19	C-20	C-21	C-22	C-23	C-24	C-25	C-26	C-27	C-17
4	C-d	C-e	C-k	C-a	C-10	C-f	C-15	C-b	C-11	C-13	C-16	C-c	C-12	C-14	C-9
5	C-28	C-29	C-30	C-28	C-4	C-31	C-32	C-29	C-31	C-5	C-33	C-30	C-32	C-33	C-3
6	C-21	C-g	C-22	C-h	C-23	C-i	C-24	C-j	C-25	C-18	C-26	C-19	C-27	C-20	C-17
7	C-35	C-36	C-37	C-38	C-39	C-40	C-41	C-42	C-43	C-44	C-45	C-46	C-47	C-48	C-34
8	C-b	C-d	C-11	C-e	C-13	C-k	C-16	C-a	C-c	C-10	C-12	C-f	C-14	C-15	C-9
9	C-h	C-j	C-19	C-21	C-23	C-25	C-27	C-g	C-i	C-18	C-20	C-22	C-24	C-26	C-17
10	C-29	C-28	C-31	C-29	C-5	C-30	C-33	C-28	C-30	C-4	C-32	C-31	C-33	C-32	C-3
11	C-36	C-38	C-40	C-42	C-44	C-46	C-48	C-35	C-37	C-39	C-41	C-43	C-45	C-47	C-34
12	C-j	C-21	C-25	C-g	C-18	C-22	C-26	C-h	C-19	C-23	C-27	C-i	C-20	C-24	C-17
13	C-38	C-42	C-46	C-35	C-39	C-43	C-47	C-36	C-40	C-44	C-48	C-37	C-41	C-45	C-34
14	C-42	C-35	C-43	C-36	C-44	C-37	C-45	C-38	C-46	C-39	C-47	C-40	C-48	C-41	C-34
0	C-6	C-6	C-7	C-6	C-2	C-7	C-8	C-6	C-7	C-2	C-8	C-7	C-8	C-8	C-1

Fig. 3. Conjugate class structure for the spectrum of a 15×15 2-D BCH code with $t = 2$. The zeros are represented in yellow cells, whereas the non-zero message positions are shown in white and green. The IPs are marked in green. Spectral nulls comprise 11 conjugate classes represented by C-a, C-a, ..., C-k. The remaining 181 message positions are segregated into 48 conjugate classes denoted by C-1, C-2, ..., C-48.

TABLE I
DIFFERENT CODE RATES POSSIBLE FOR A 15×15 2-D BCH CODE WITH A QUASI-CYCLIC BURST ERROR CORRECTION CAPABILITY $t \times t$

Burst size $(t \times t)$	Number of spectral nulls	Code Rate
2×2	44	$\frac{181}{225}$
3×3	94	$\frac{131}{225}$
4×4	154	$\frac{71}{225}$
5×5	176	$\frac{49}{225}$
6×6	192	$\frac{33}{225}$
7×7	196	$\frac{29}{225}$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.



Field element	Binary mappping
0	0000
1	1000
α^1	0100
α^2	0010
α^3	0001
α^4	1100
α^5	0110
α^6	0011
α^7	1101
α^8	1010
α^9	0101
α^{10}	1110
α^{11}	0111
α^{12}	1111
α^{13}	1011
α^{14}	1001

(b)

j,j'	1	2	3	4	5	6	7	8	9	10	11	12	13	14	0
1	0	0	0	0	α^{14}	α^3	α^4	0	0	α^9	α^2	0	α^{14}	α^{13}	α^0
2	0	0	0	0	α^3	0	α^4	0	0	α^{13}	α^{13}	α^6	α^{11}	α^8	α^0
3	0	0	0	0	α^8	0	α^2	α^4	α^1	α^0	α^{10}	α^2	α^{14}	α^1	α^3
4	0	0	0	0	α^{11}	0	α^{11}	0	α^{12}	α^6	α^7	0	α^1	α^8	α^0
5	α^7	α^0	α^7	α^{13}	0	0	α^3	α^0	0	α^5	α^5	α^{13}	α^{12}	α^5	α^0
6	α^8	0	α^2	0	α^0	0	α^5	0	α^4	α^1	α^{13}	0	α^2	α^4	α^6
7	α^1	α^{11}	α^3	α^1	α^8	α^5	α^8	α^{12}	α^0	α^{12}	α^9	α^{11}	α^4	α^{11}	α^2
8	0	0	α^9	0	α^{12}	0	α^{14}	0	0	α^7	α^2	0	α^1	α^7	α^0
9	0	0	0	α^2	α^0	α^1	α^8	0	0	α^4	α^1	α^8	α^5	α^7	α^9
10	α^0	α^{14}	0	α^0	α^{10}	α^{14}	α^{10}	α^{11}	α^{11}	0	α^9	0	α^{10}	α^6	α^0
11	α^{13}	α^8	α^{10}	α^6	α^6	α^{13}	α^{13}	α^8	α^9	α^4	α^4	α^0	α^{12}	α^2	α^1
12	0	α^1	α^8	0	α^2	α^4	α^{11}	0	0	α^0	α^4	0	α^8	α^{10}	α^{12}
13	α^4	α^3	α^{14}	α^4	α^2	α^0	α^1	α^{14}	α^5	α^3	α^{14}	α^{12}	α^2	α^6	α^8
14	α^9	α^2	α^0	α^7	α^9	α^6	α^3	α^2	α^7	α^1	α^8	α^{10}	α^7	α^1	α^4
0	α^1	α^2	α^2	α^4	α^5	α^4	α^9	α^8	α^1	α^{10}	α^{12}	α^8	α^6	α^3	α^0

(c)

Fig. 4. (a) Random binary message vector of length 181 was divided into sub-messages depending on the length of the conjugate class it is encoded into. (b) Mapping of field elements of GF(2^4) to a 4-bit binary vectors. (c) Complete encoded spectrum for a 15×15, $t = 2$, 2-D BCH code. Yellow cells represent spectral nulls. The sub-messages are mapped into elements from GF(2^4) and encoded into the respective IPs denoted in green. The rest of the positions are determined through conjugacy constraints.

B. Reduction of Algorithmic Complexity and Hardware Architecture of the Encoder

We have the 2-D IDFFFT from (1b) for $n = n'$ and $q = 2$ as

$$c_{i,i'} = \sum_{j=0}^{n-1}\sum_{j'=0}^{n-1} \alpha^{-ij}\alpha^{-i'j'} C_{j,j'} = \sum_{j=0}^{n-1}\sum_{j'=0}^{n-1} \alpha^{-(ij+i'j')} C_{j,j'}. \quad (6)$$

For a certain IDFFFT point (i, i'), let $[\hat{C}_{j,j'}] = [\alpha^{-(ij+i'j')}]$. Now, let $[\overline{C}_{j,j'}] = [C_{j,j'}] \circ [\hat{C}_{j,j'}]$ be the Hadamard product (i.e., point-wise multiplication of elements) of $[C_{j,j'}]$ and $[\hat{C}_{j,j'}]$. Using this, we can re-write (6) as[2]

$$c_{i,i'} = \sum_{j=0}^{n-1}\sum_{j'=0}^{n-1} \overline{C}_{j,j'}. \quad (7)$$

From (7), we observe that adding all elements in the $n \times n$ array $[\overline{C}_{j,j'}]$ results in the 2-D IDFFFT at a certain point (i, i'). We know that by construction, the spectrum $[C_{j,j'}]$ has a particular conjugate class structure. We show in the following theorem that the elements of $[\hat{C}_{j,j'}]$ and the Hadamard product $[\overline{C}_{j,j'}]$ are also segregated into the same conjugate class structure.

Theorem 1: The elements of $[\hat{C}_{j,j'}]$ and the Hadamard product $[\overline{C}_{j,j'}]$ are grouped into the same conjugate class structure as $[C_{j,j'}]$.

[2]Unless otherwise mentioned, $[c_{i,i'}]$, $[C_{j,j'}]$, $[\overline{C}_{j,j'}]$, and $[\hat{C}_{j,j'}]$ are arrays of size $n \times n$ throughout this article.

Proof: Let the field element at a certain position (j, j') in the spectrum $[C_{j,j'}]$ be α^k, which belongs to the conjugate class $\{\alpha^k, \alpha^{2k}, \alpha^{4k}, \ldots, \alpha^{2^{m-1}k}\}$ at positions $\{(j, j'), (2j, 2j'), (4j, 4j'), \ldots, (2^{m-1}j, 2^{m-1}j')\}$, respectively. The corresponding entries in the matrix $[\hat{C}_{j,j'}]$ are, respectively, $\{\alpha^{-(ij+i'j')}, \alpha^{-(i\cdot 2j+i'\cdot 2j')}, \alpha^{-(i\cdot 4j+i'\cdot 4j')}, \ldots, \alpha^{-(i\cdot 2^{m-1}j+i'\cdot 2^{m-1}j')}\}$. This can also be written as $\{\alpha^{-(ij+i'j')}, (\alpha^{-(ij+i'j')})^2, (\alpha^{-(ij+i'j')})^4, \ldots, (\alpha^{-(ij+i'j')})^{2^{m-1}}\}$. It can be clearly seen that this has the same conjugate class structure as $[C_{j,j'}]$.

By multiplying the elements of $[C_{j,j'}]$ and $[\hat{C}_{j,j'}]$ at positions $\{(j, j'), (2j, 2j'), (4j, 4j'), \ldots, (2^{m-1}j, 2^{m-1}j')\}$, we get the corresponding elements of $[\overline{C}_{j,j'}]$ as $\{\alpha^k\alpha^{-(ij+i'j')}, \alpha^{2k}(\alpha^{-(ij+i'j')})^2, \alpha^{4k}(\alpha^{-(ij+i'j')})^4, \ldots, \alpha^{2^{m-1}k}(\alpha^{-(ij+i'j')})^{2^{m-1}}\}$. Upon rearrangement, we observe that the elements of $[\overline{C}_{j,j'}]$ at points $\{(j, j'), (2j, 2j'), (4j, 4j'), \ldots, (2^{m-1}j, 2^{m-1}j')\}$ are, respectively, $\{\alpha^{-(ij+i'j-k')}, (\alpha^{-(ij+i'j'-k)})^2, (\alpha^{-(ij+i'j'-k)})^4, \ldots, (\alpha^{-(ij+i'j'-k)})^{2^{m-1}}\}$. It is easy to observe that this has the same conjugate class structure as $[C_{j,j'}]$ and $[\hat{C}_{j,j'}]$. Thus, we have proved that the elements of $[\hat{C}_{j,j'}]$ and the Hadamard product $[\overline{C}_{j,j'}]$ are grouped into the same conjugate class structure as $[C_{j,j'}]$. ■

We shall now use Theorem 1 to reduce the complexity of 2-D IDFFFT unit. We make the following observations.

Observation 1: Let $P_1(\alpha)$ and $P_2(\alpha)$ be the polynomial representations of the elements at a certain point (j, j') of $[C_{j,j'}]$ and $[\hat{C}_{j,j'}]$, respectively. Thus, the element at (j, j') in $[\overline{C}_{j,j'}]$ is $P_1(\alpha)P_2(\alpha)$. Computing 2-D IDFFFT at a certain point (i, i') involves multiplying $P_1(\alpha)$ and $P_2(\alpha)$ at all k message

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

positions of the spectrum and adding the resulting $P_1(\alpha)P_2(\alpha)$ obtained at all k positions of $[\overline{C}_{j,j'}]$. Theorem 1 states that the elements of $[\overline{C}_{j,j'}]$ has the same conjugate class structure as $[C_{j,j'}]$. Thus, the elements of $[\overline{C}_{j,j'}]$ are related through (2), allowing us to reduce the 2-D IDFFFT computation complexity at (i,i') from k points to 1 point per conjugate class in $[\overline{C}_{j,j'}]$, which we call the IP. Using (2), we obtain the rest of the elements by taking 2nd, 4th, 8th, ..., (2^{m-1})th powers of the element at the IP.

Observation 2: Only the MSB of the polynomial product $P_1(\alpha)P_2(\alpha)$ at the each IP of $[\overline{C}_{j,j'}]$ needs to be found. This is because 2-D IDFFFT always results in a binary array due to (2).

Using the above observations, we reduce the hardware complexity of the encoder significantly. We show the encoding process for a 15×15, $t = 2$ 2-D BCH code to illustrate the complexity reduction, but the procedure can be applied for a general $n \times n$ BCH code with different values of t.

Example: We take an extension field GF(2^4), using the primitive polynomial $p(x) = x^4 + x + 1 = 0$. Considering $P(\alpha) = a_0 + a_1\alpha + a_2\alpha^2 + a_3\alpha^3$ as the polynomial representation of an element of a conjugate class, the other elements can be generated using the following equation:

$$\begin{aligned} P^2(\alpha) &= (a_0 + a_2)\alpha^0 + a_2\alpha^1 + (a_1 + a_3)\alpha^2 + a_3\alpha^3. \\ P^4(\alpha) &= (P^2(\alpha))^2 = (a_0 + a_1 + a_2 + a_3)\alpha^0 + (a_1 + a_3)\alpha^1 \\ &\quad + (a_2 + a_3)\alpha^2 + a_3\alpha^3. \\ P^8(\alpha) &= (P^4(\alpha))^2 = (a_0 + a_1)\alpha^0 + (a_2 + a_3)\alpha^1 + a_1\alpha^2 \\ &\quad + a_3\alpha^3. \end{aligned} \tag{8}$$

Let the polynomial representations of two elements $P_1(\alpha)$ and $P_2(\alpha)$ from GF(2^4) be $P_1(\alpha) = a_0 + a_1\alpha + a_2\alpha^2 + a_3\alpha^3$ and $P_2(\alpha) = b_0 + b_1\alpha + b_2\alpha^2 + b_3\alpha^3$. Multiplying $P_1(\alpha)$ and $P_2(\alpha)$, we get

$$\begin{aligned} &P_1(\alpha)P_2(\alpha) \\ &\quad = \alpha^0(a_0b_0 + a_1b_3 + a_2b_2 + a_3b_1) + \alpha^1 \\ &\qquad \times (a_0b_1 + a_1b_0 + a_1b_3 + a_2b_2 + a_3b_1 + a_2b_3 + a_3b_2) \\ &\qquad + \alpha^2(a_0b_2 + a_1b_1 + a_2b_0 + a_2b_3 + a_3b_2 + a_3b_3) \\ &\qquad + \alpha^3(a_0b_3 + a_1b_2 + a_2b_1 + a_3b_0 + a_3b_3). \\ &\quad = c_0 + c_1\alpha + c_2\alpha^2 + c_3\alpha^3. \end{aligned} \tag{9}$$

For encoding the message bits, we have 1, 4, and 43 conjugate classes of sizes 1, 2, and 4, respectively, as shown in Fig. 3. For a size 4 conjugate class, the MSB of sum of the polynomial product $P_1(\alpha)P_2(\alpha)$ at the four points is

$$\begin{aligned} \text{out}^{\text{MSB}}_{\text{size 4}} &= c_0 \oplus (c_0 \oplus c_2) \oplus (c_0 \oplus c_1 \oplus c_2 \oplus c_3) \oplus (c_0 \oplus c_1) \\ &= c_3 = a_0b_3 + a_1b_2 + a_2b_1 + a_3b_0 + a_3b_3. \end{aligned} \tag{10}$$

Similarly, for a size 2 conjugate class, the MSB of sum of the polynomial product $P_1(\alpha) \times P_2(\alpha)$ at the two points is

$$\begin{aligned} \text{out}^{\text{MSB}}_{\text{size 2}} &= c_0 \oplus (c_0 \oplus c_2) = c_2 \\ &= a_0b_2 + a_1b_1 + a_2b_0 + a_2b_3 + a_3b_2 + a_3b_3. \end{aligned} \tag{11}$$

Thus, MSB of the sum of elements for conjugate classes of sizes 4 and 2 is, respectively, the LSB and the second LSB of the element at the IP.

TABLE II
REDUCTION IN HARDWARE REQUIREMENTS AS A RESULT OF THE PROPOSED COMPLEXITY REDUCTION FOR A 15×15, $t = 2$ 2-D BCH CODE GF(2^4)

Gates for 2-D IDFFFT at (i,i')	Using complexity reduction	Brute force (Over all 181 message points)
AND	$4 \times 6 + 43 \times 5 = 239$	$181 \times 22 = 3982$
XOR	$4 \times 5 + 43 \times 4 + 47 = 239$	$181 \times 18 + 180 \times 4 = 3978$

The multiplier complexity was reduced to five AND gates and four XOR gates per IP for a size four conjugate class and six AND gates and five XOR gates for a size 2 conjugate class. In comparison, without reduction, we need 22 AND gates and 18 XOR gates for $k = 181$ points. The reduction in number of gates is summarized in Table II, which shows a reduction in the number of gates by ~94% per IDFFFT point (i,i').

C. Proposed Architecture of the Encoder

Fig. 5(a) shows the proposed architecture of the 2-D BCH encoder. It consists of a message bits to sub-messages converter that divides the message bits into sub-messages $S^{\text{IP}}_{\text{cc}_l}$, depending on the size of the conjugate class. The architecture consists of $n \times n$ parallel IDFFFT blocks that compute the 2-D IDFFFT for all (i,i'), $0 \le i, i' \le n-1$.

The IDFFFT block shown in Fig. 5(b) consists of cc_{en} parallel conjugate class blocks, where cc_{en} is the number of conjugate classes used for encoding messages. Subsequently, a XOR array is used to XOR all the outputs of the cc_{en} conjugate class blocks to produce the 2-D IDFFFT at (i,i').

Each conjugate class unit has two inputs, $(\alpha^{-(ij+i'j')})^{\text{IP}}_{\text{cc}_l}$ and $S^{\text{IP}}_{\text{cc}_l}$ corresponding to the elements in $[\hat{C}_{j,j'}]$ and $[C_{j,j'}]$, respectively. IP denotes the IP and cc_l denotes the conjugate class index. For a certain conjugate class block in a particular IDFFFT unit, the values of i, i', j, j' for $(\alpha^{-(ij+i'j')})^{\text{IP}}_{\text{cc}_l}$ are fixed and already known during the design. Thus, instead of using a circuit to evaluate the value of $(\alpha^{-(ij+i'j')})^{\text{IP}}_{\text{cc}_l}$, we hard-code the values of $(\alpha^{-(ij+i'j')})^{\text{IP}}_{\text{cc}_l}$ for each conjugate class input in all the n^2 IDFFFT block. This reduces the hardware requirement significantly.[3] The conjugate class unit finds the MSB of the polynomial product using a combinational logic based on the reduced architecture explained in Section III-B. At the end of a clock cycle, we have the 2-D IDFFFT for all (i,i') positions, i.e., an $n \times n$ array of elements from GF(2), which is the encoded message.

IV. DECODING PROCEDURE AND HARDWARE ARCHITECTURE

In this section, we first discuss the decoding algorithm IBA-I [15]. Next, we propose modifications to IBA-I to reduce

[3]To illustrate this, we take the example in Section III-B. We observe in Figs. 3 and 4 that IP for $\text{cc}_l = 41$ corresponds to $j = 7, j' = 7$, which gives $S^{\text{IP}}_{41} = \alpha^8$. For evaluating the IDFFFT at a certain point $i = 2, i' = 3$, for the conjugate class block $\text{cc}_l = 41$, we have $i = 2, i' = 3, j = 7, j' = 7$. Thus, we have $(\alpha^{-(ij+i'j')})^{\text{IP}}_{\text{cc}_l} = \alpha^{-(14+21)} = \alpha^{10}$. The polynomial product of α^8 and α^{10} needs to be evaluated.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

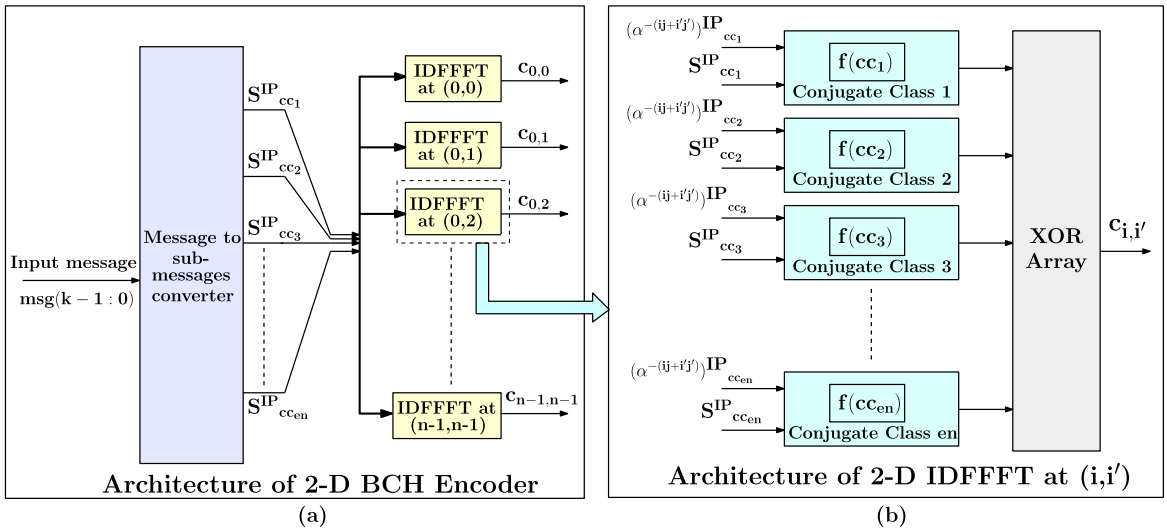


Fig. 5. (a) Overall architecture of the encoder contains n^2 units to compute the 2-D IDFFFT at all (i, i'). (b) Architecture of the IDFFFT block consists of cc_{en} parallel units to compute the 2-D IDFFFT at a certain point (i, i'). Conjugate class unit uses the reduced architecture, as explained in Section III-B.

the complexity of the DFFFT and IDFFFT units and allow parallel implementations of the BM units. Subsequently, we present the proposed decoder hardware architecture.

A. Decoding Algorithm

Madhusudhana and Siddiqi [15] proposed IBA-I for correction of cyclic and quasi-cyclic bursts in an $n \times n$ 2-D BCH code, with a zero structure given by (5). Let us consider an $n \times n$ 2-D BCH code $[c]$ encoded over a channel. An $n \times n$ array $[r]$ is received with an additive error $[e]$, i.e., $[r] = [c] + [e]$. The reader must note that in post DDNP (data-dependent noise prediction) and ISI detection, the signal chain collapses to an additive error model for decoding the codeword [18]. Blahut's algorithm BA-I can be summarized as follows. The 2-D DFFFT of the received codeword is taken, giving the syndromes at the $2t \times 2t$ positions, i.e., the spectral null locations. BM engine is performed on the $2t$ columns[4] to find the connection polynomial for the columns, which is used for RE to find the remaining $2t \times (n - 2t)$ syndromes over the $2t$ columns. Similarly, the BM engine is performed on the n rows to find the connection polynomial for the rows, which is used for RE to find the remaining $n \times (n - 2t)$ syndromes for the n rows.

The steps of the IBA-I are summarized in Algorithm 2.

B. Algorithmic Complexity Reduction

We note the following in IBA-I (Algorithm 2).

1) In step 1, only the elements in the $2t \times 2t$ block of $[R]$, i.e., $[R_{j,j'}]_{j,j'=1}^{2t}$, are required for steps 2–5. Thus, it is sufficient to evaluate the DFFFT at these $2t \times 2t$ positions. This will require modifying step 6, where the IDFFFT needs to be performed over the error word array $[e]$, and subsequently, one should perform $[c] = [e]+[r]$ in the time domain to get the corrected codeword.
2) Some of the syndromes in $[S_{j,j'}]_{j,j'=1}^{2t}$ are related to each other through the conjugacy constraints. This can further reduce the number of DFFFT computations. For example, in Fig. 3, it can be seen that within the $4 \times 4 = 16$ zero positions in the 15×15 codeword, there are 11 independent syndromes and the rest are related to them by conjugacy constraints. Thus, we can reduce the number of DFFFT points from $2t \times 2t$ to number of conjugate classes used for spectral nulls.
3) Instead of processing the BM engine serially along the rows (columns) to evaluate the common connection polynomial, we can perform the BM engines along rows (columns) in parallel. This will require finding the common connection polynomial by taking the least common multiple (lcm) of connection polynomials found over the rows (columns) as mentioned in [15, Remark 7].
4) It is mentioned in [15, Remark 6] that if p and p' are conjugates with respect to n, pth and p'th row (column) will satisfy the same row (column) connection polynomial. Thus, it is sufficient to perform BM engine on the rows (columns) that are not conjugates of each other. This will significantly reduce the number of passes through the BM engine and also reduce the complexity of the common connection polynomial finder as lcm needs to be taken only for the connection polynomials for the rows (columns) that are not conjugates of each other with respect to n.
5) The 2-D IDFFFT unit can be designed similar to the design of the encoder as described in Section III-B, reducing the number of gates. We will discuss more about it in Section IV-D.

C. Modified Decoding Algorithm

The modified decoding algorithm employing the complexity reductions in Section IV-B can be summarized in Algorithm 3.

The above algorithm reduces the complexities of DFFFT and IDFFFT units and reduces latency significantly by

[4]The BM engine may stop once we achieve a degree t polynomial for a certain column (row).

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Steps of the Decoding Algorithm IBA-I [15]

Input: Received codeword $[r]$
Output: Corrected codeword $[c]$
Outline of steps:
Step 1 - 2-D DFFFT computation: Obtain the 2-D DFFFT of the received codeword $[r]$ to obtain $[R]$. Set the syndrome array as follows

$$\left[S_{j,j'}\right]_{j,j'=1}^{2t} = \left[R_{j,j'}\right]_{j,j'=1}^{2t} \tag{12}$$

Step 2 - BM engine along the columns: Run the BM engine serially along columns $j' = 1$ to $2t$. For a fixed column j', the BM engine is performed using the syndromes $[S_{j,j'}]$ for $j = 1$ to $2t$, with the initialization $c_{\text{init}}^{(j')}(x) = c_{\text{fin}}^{(j'-1)}(x)$, where the subscript 'init' in $c_{\text{init}}^{(j')}(x)$ represents the initialization for iteration 1 of the BM engine for the j'^{th} column and subscript 'fin' in $c_{\text{fin}}^{(j'-1)}(x)$ represents the output of the BM engine for the $(j'-1)^{\text{st}}$ column. For column 1, the initialization is done with $c_{\text{init}}^{(1)}(x) = 1$. This means that the connection polynomial for $(j'-1)^{\text{st}}$ column is used as the initialization for j'^{th} column. When the $2t$ columns have been processed, we achieve the common connection polynomial for the columns given by

$$c^{(\text{ccp})}(x) = c_0^{(\text{ccp})} + c_1^{(\text{ccp})}x + \cdots + c_t^{(\text{ccp})}x^t \tag{13}$$

Step 3 **- Recursive extension along the columns:** Using $c^{(\text{ccp})}(x)$, perform recursive extension on columns $j' = 1$ to $2t$ using equation (14) to evaluate the remaining syndromes for columns 1 to $2t$.

$$S_{j,j'} = \sum_{\mu=1}^{t} c_\mu^{(\text{ccp})} S_{\text{mod}(j-\mu,n),j'} \tag{14}$$

for $j' = 1, 2, \ldots, 2t$ and $j = 0, 2t+1, \ldots, n-1$.
Step 4 **- BM engine along the rows:** Similar to Step 2, BM engine is performed serially along the rows $j = 1$ to $2t$ to produce the common connection polynomial for the rows:

$$r^{(\text{ccp})}(x) = r_0^{(\text{ccp})} + r_1^{(\text{ccp})}x + \cdots + r_t^{(\text{ccp})}x^t \tag{15}$$

Step 5 **- Recursive extension along the rows:** Using $r^{(\text{ccp})}(x)$, perform recursive extension on rows $j = 0$ to $n-1$ using equation (16) to evaluate the remaining syndromes for rows 0 to $n-1$.

$$S_{j,j'} = \sum_{\mu=1}^{t} r_\mu^{(\text{ccp})} S_{j,\text{mod}(j'-\mu,n)} \tag{16}$$

for $j = 0, 1, \ldots, n-1$ and $j' = 0, 2t+1, \ldots, n-1$. This produces the complete syndrome array $[S_{j,j'}]_{j,j'=0}^{n-1}$, which is the DFFFT of $[e]$.
Step 6 **- Corrected codeword in the frequency domain:** The corrected codeword in the frequency domain is given by

$$C_{j,j'} = R_{j,j'} + S_{j,j'}; \quad 0 \le j,\ j' \le n-1 \tag{17}$$

Step 7 **- 2-D IDFFFT computation:** 2-D IDFFFT of $[C]$ gives the corrected codeword in the time domain, i.e., $[c]$.

Algorithm 3: Modified Decoding Algorithm (Parallel Implementation)

Input: Received codeword $[r]$
Output: Corrected Codeword $[c]$
Outline of steps:
Step 1 **- 2-D DFFFT computation:** Obtain the syndrome array $[S_{j,j'}]_{j,j'=1}^{2t}$ by evaluating the DFFFT at cc_z points in the $2t \times 2t$ block, where cc_z is the number of zeros used for spectral nulls. The DFFFT at the remaining $2t \times 2t - cc_z$ points are evaluated from the cc_z points through the conjugacy constraints in equation (2). This gives the syndrome array $[S_{j,j'}]_{j,j'=1}^{2t}$.
Step 2 **- BM engine along the columns:** For columns $j' = \{p_1, p_2, \cdots, p_{c_p}\}$, BM engine is performed in *parallel* using the syndromes $[S_{j,j'}]_{j=1}^{2t}$. Here $\{p_1, p_2, \cdots, p_{c_p}\}$ denote the indices of the columns among $j' = 1$ to $2t$ that are not conjugates of each other w.r.t. n and c_p denotes the number of non-conjugate columns. The common connection polynomial for columns i.e., $c^{(\text{ccp})}(x)$ is evaluated by taking lcm of the connection polynomials obtained from the individual BM engines.
Step 3 **- Recursive extension along the columns:** This is same as in Step 3 of Algorithm 2.
Step 4 **- BM engine along the rows:** For rows $j = \{p_1, p_2, \cdots, p_{c_p}\}$, BM engine is performed in *parallel* using the syndromes $[S_{j,j'}]_{j'=1}^{2t}$. Here $\{p_1, p_2, \cdots, p_{c_p}\}$ denote the indices of the rows among $j = 1$ to $2t$ that are not conjugates of each other w.r.t. n and c_p denotes the number of non-conjugate rows. The common connection polynomial for rows is evaluated by taking lcm of the connection polynomials obtained from the individual BM engines.
Step 5 **- Recursive extension along the rows:** This is same as in Step 5 of Algorithm 2. This generates the complete syndrome array $[S_{j,j'}]_{j,j'=0}^{n-1}$.
Step 6 **- 2-D IDFFFT computation:** Perform IDFFFT of $[S_{j,j'}]_{j,j'=0}^{n-1}$ to get error array in time domain, i.e. $[e]$.
Step 7 **- Corrected codeword in the time domain:** $[c] = [e] + [r]$ gives the corrected codeword array in time domain.

facilitating parallel implementations of the BM engines. We illustrate the decoding process with the help of an example as shown in the following.

Example: Let us consider a 2-D BCH code of size 5×5 with a burst error correction capability 2×2. The zeros of the code $[C_{j,j'}]$ are $1 \le j, j' \le 4$, where j is along the row direction and j' is along the column direction. We consider a zero codeword transmitted in the time domain. Let the channel introduce a quasi-cyclic burst given by $e(x, y) = x + xy^3 + x^3y^3$. The primitive polynomial $P(z) = z^4 + z + 1$ is used for generating GF(2^4) with $\alpha = z$ as the primitive element. Evaluating the DFFFT at $1 \le j, j' \le 4$, we get the syndrome array available for processing given in Fig. 6(b). Since numbers 1–4 are conjugates with respect to 5, rows 1–4 are conjugates of each other. Thus, it follows from step 4 of Algorithm 3 that

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

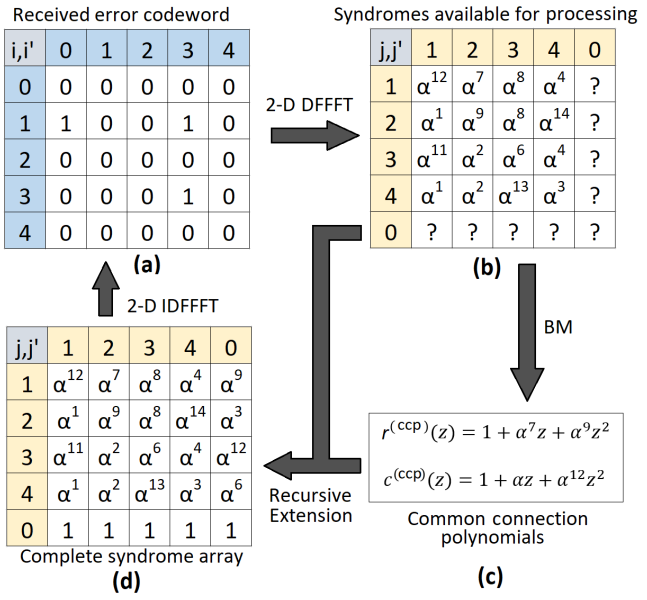


Fig. 6. Decoding example using a BCH code of size 5×5 with $t = 2$. (a) Received error codeword. (b) Syndromes available for processing. "?" represents the unknown positions in the spectral domain. (c) Common connection polynomials for row and column. (d) Complete syndrome array produced by RE.

the BM engine needs to be performed only on one of those rows. Applying BM engine on row 1, we get the common row connection polynomial $r^{(\mathrm{ccp})}(z) = r_0^{(\mathrm{ccp})} + r_1^{(\mathrm{ccp})} z + r_2^{(\mathrm{ccp})} z^2 = 1 + \alpha^7 z + \alpha^9 z^2$. This can be used to perform RE along the rows given by equation $S_{j,0} = r_1^{(\mathrm{ccp})} S_{j,4} + r_2^{(\mathrm{ccp})} S_{j,3}$ for $1 \leq j \leq 4$, as shown in Fig. 6(d). Similarly, columns 1–4 are conjugates of each other. Thus, we perform BM engine on only column 1 to get the common column connection polynomial $c^{(\mathrm{ccp})}(z) = c_0^{(\mathrm{ccp})} + c_1^{(\mathrm{ccp})} z + c_2^{(\mathrm{ccp})} z^2 = 1 + \alpha z + \alpha^{12} z^2$. This is used to perform RE along the columns given by equation $S_{0,j'} = c_1^{(\mathrm{ccp})} S_{4,j'} + c_2^{(\mathrm{ccp})} S_{3,j'}$ for $0 \leq j' \leq 4$, as shown in Fig. 6(d). Finally, we perform the 2-D IDFFFT on the complete syndrome array $[S_{j,j'}]_{j,j'=0}^{4}$ to get $[e]$, as shown in Fig. 6(a).

D. Code Failure Analysis

As mentioned before, post DDNP and ISI detection, the read channel in magnetic storage is equivalent to an additive white Gaussian noise (AWGN) channel. Thus, we can emulate the coded system over an AWGN channel to evaluate the performance. We emulate the 2-D BCH system for a size 15×15 and $t = 2$ on MATLAB to evaluate the bit error rates (BERs) and code failure rates (CFRs) at different signal-to-noise ratios (SNRs), accounting for the code rate penalty using Monte Carlo simulations. We also compare the performance of the above code with a product of two $(15, 7)$ BCH code with $t = 2$ having a row decoder followed by a column decoder.

The CFR performance of the 2-D BCH code and BCH product codes with code parameters given in the last paragraph is shown in Fig. 7(a). We notice that the 2-D BCH code has about 0.5 dB better performance than the BCH product code at a CFR of 10^{-4}. Similarly, the BER performance of the 2-D BCH code and BCH product code is shown in Fig. 7(b). We

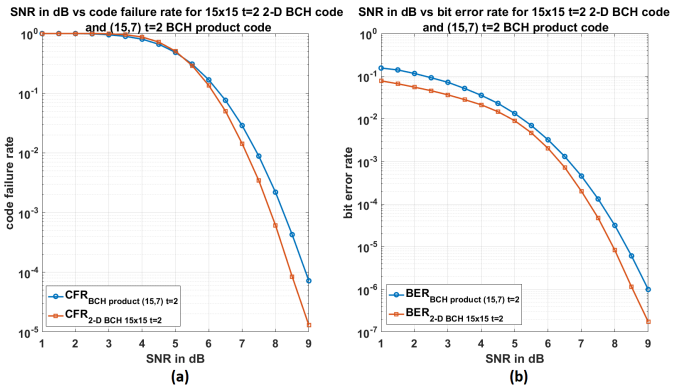


Fig. 7. (a) SNR in dB versus CFR for 15×15, $t = 2$ 2-D BCH codes and $(15, 7)$ BCH product code with $t = 2$. (b) SNR in dB versus BER for 15×15, $t = 2$ 2-D BCH codes, and $(15, 7)$ BCH product code with $t = 2$.

observe a performance gain of about 0.5 dB achieved by 2-D BCH codes over the BCH product codes at a BER of 10^{-5}.

E. Hardware Architecture of the Decoder

The top-level architecture of the decoder for a 2-D BCH code of size $n \times n$ with burst error correction capability $t \times t$ is given in Fig. 8. It has six pipelined stages, where each stage is separated using pipeline registers. The received codeword is fed into the *2-D DFFFT finder,* which evaluates the syndrome array $[S_{j,j'}]_{j,j'=1}^{2t}$ by performing DFFFT over $2t \times 2t$ points of the syndrome array. The outputs of the 2-D DFFFT finder are fed to parallel stages of the *BM engine* which evaluate the connection polynomials for the $2t$ rows and $2t$ columns. As mentioned in step 2 in Section IV-C, $2 \times c_p$ parallel BM engines are required. The multiplier 2 appears as c_p BM engines are required for processing the rows and c_p BM engines are required for processing the columns.

Subsequently, a common connection polynomial finder evaluates the common column and common row connection polynomials used to perform RE over columns and rows. Using the common column connection polynomial, $2t$ parallel stages of RE unit are used to perform RE over $2t$ columns. Next, using the common row connection polynomial, n parallel stages of RE unit are used to perform RE over n rows. Now, we have the complete syndrome array given by $[S_{j,j'}]_{j,j'=0}^{n-1}$. Subsequently, the 2-D IDFFFT finder uses the reduced complexity algorithm as described in Section III-B to evaluate the 2-D IDFFFT of $[S_{j,j'}]_{j,j'=0}^{n-1}$ to generate the error array $[e]$, which is XORed with the received codeword $[r]$ to produce the corrected codeword $[c]$. The hardware architectures of the pipelined stages are explained in detail as follows.

1) 2-D DFFFT Finder: The hardware architecture of the 2-D DFFFT finder is shown in Fig. 9(a). It consists of $2t \times 2t$ parallel DFFFT units that compute the 2-D DFFFT in parallel at (j, j') for $1 \leq j, j' \leq 2t$. If a set of points $(j_x, j'_x)_{x=1}^{l}$ in $1 \leq j_x, j'_x \leq 2t$ are conjugates of each other with respect to n, the DFFFT needs to be evaluated only at 1 point per conjugate class, as mentioned in [15, Remark 6]. Thus, the DFFFT units at the other points will have a power finder, which will find the appropriate power according to (2). The power

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

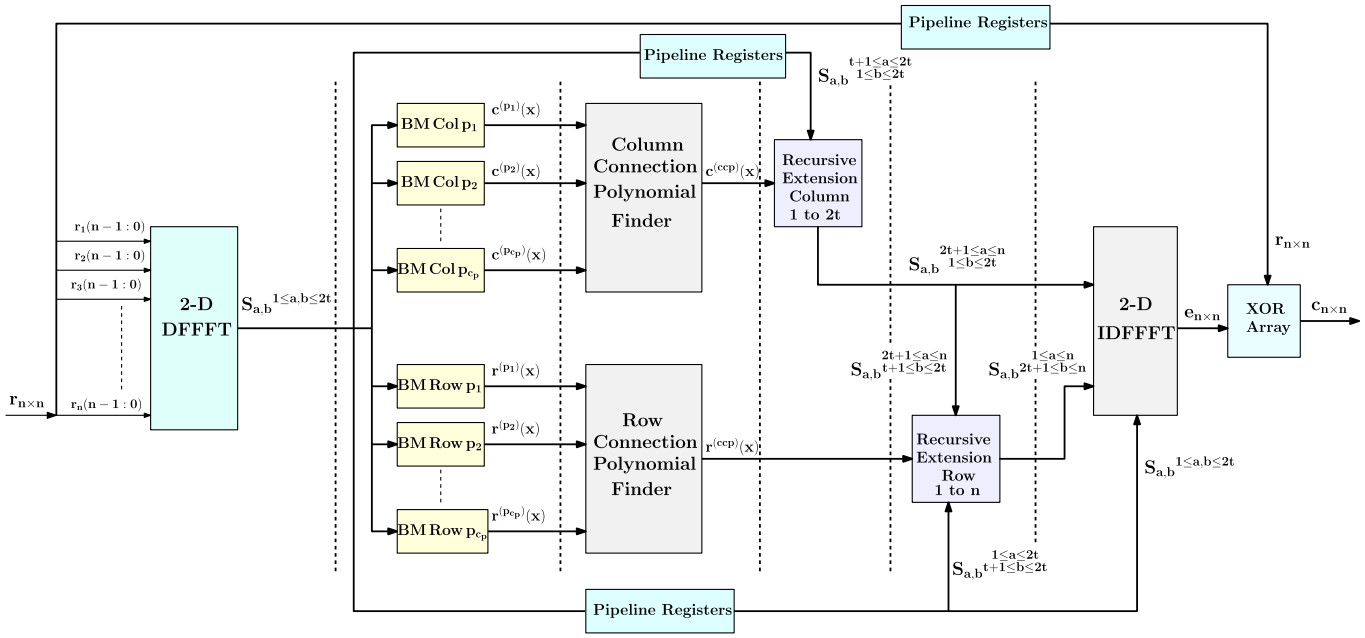


Fig. 8. Top-level architecture of the BCH decoder. It consists of six pipelined stages, namely the 2-D DFFFT, BM engine, connection polynomial finder, RE along rows, RE along columns, and the 2-D IDFFFT unit. The stages take 1, $2t$, 1, 1, 1, and 1 clock cycles. The decoder is a pipelined architecture with a latency of $2t + 5$, able to process a codeword every $2t$ clock cycles. Each stage is separated using pipeline registers.

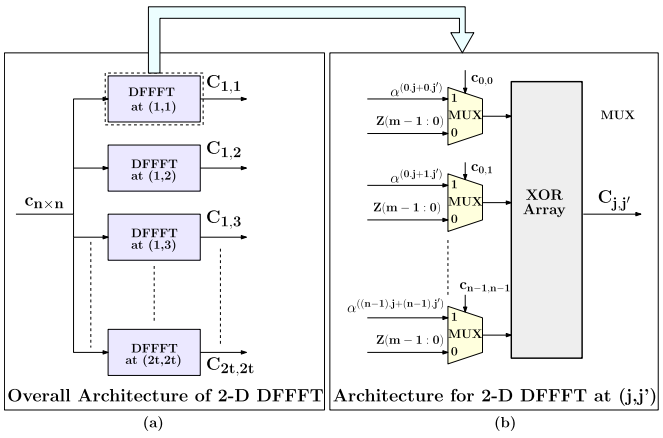


Fig. 9. (a) Hardware architecture of the 2-D DFFFT unit. It consists of $2t \times 2t$ parallel DFFFT units that compute the 2-D DFFFT at (j, j') for $1 \le j, j' \le 2t$. (b) Each DFFFT unit consists of n^2 parallel multiplexers that calculate the DFFFT at a certain point using MUX-es and a XOR array.

finder architecture can easily be designed according to the combinational logic in (8).

Each DFFFT unit shown in Fig. 9(b) consists of n^2 parallel multiplexers which choose either $\alpha^{(ij+i'j')}$ or a m-bit zero vector $Z(m-1{:}0)$, depending on the position (i, i') of the input binary array $[c_{i,i'}]$. A XOR array is used to XOR the n^2 outputs of the multiplexers to produce the DFFFT at point (j, j') in a single clock cycle.

2) BM Engine: For details of the BM algorithm, the reader is encouraged to go through [19]. The BM engine finds the discrepancy factor (d_μ) and the correction factor $d_\mu d_\rho^{-1} x^{(\mu-\rho)} \sigma^{(\rho)}(x)$, which will be used to update $\sigma^{(\mu+1)}(x)$ for the next iteration. We use an architecture similar to [1] and [20], as shown in Fig. 10. Registers $\sigma^{(\mu)}$ and $\sigma^{(\rho)}$ are used to store coefficients of $\sigma^{(\mu)}(x)$ and $\sigma^{(\rho)}(x)$, respectively.

TABLE III
COMPUTATIONS FOR ITERATIVE BM ALGORITHM

μ	$\sigma^{(\mu)}(x)$	d_μ	l_μ	$\mu - l_\mu$
-1	1	1	0	-1
0	1	S_1	0	0
1	$1 + S_1 x$			
⋮	⋮			
2t				

In addition, we need d_μ, d_ρ, l_μ, and l_ρ. For initialization, $\sigma^{(\rho)}$, d_ρ, and l_ρ are loaded for $\mu = -1$ from Table III. Similarly, $\sigma^{(\mu)}$ and l_μ are loaded with values corresponding to $\mu = 0$. As the decoding iterations proceed, $\sigma^{(\mu)}$ and l_μ get updated when $d_\mu \neq 0$, with values corresponding to the BM engine update equation [19]. $\sigma^{(\rho)}$, d_ρ, and l_ρ are updated with $\sigma^{(\mu)}$, d_μ, and l_μ, respectively, when $d_\mu \neq 0$ and $\mu - l_\mu \geq \rho - l_\rho$.

A "temporary register" of size $t + 1$ initialized to $S_2, S_1, 0, 0, \ldots, 0$ is used to update d_μ at each iteration. As decoding iterations proceed, the "temporary register" is loaded with a value from second index of the "Syndrome register," which is rotated as shown in Fig. 10. Thus, the "temporary register" acts as a shift register, yielding d_μ after the adder block $\sum_1$. We shall see later that the output of adder block $\sum_2$ is $\sigma^{(\mu)} + d_\mu d_\rho^{-1} x^{(\mu-\rho)} \sigma^{(\rho)}$. MUX 2 is used to update $\sigma^{(\mu)}$ register based on d_μ.

The correction factor $d_\mu d_\rho^{-1} x^{(\mu-\rho)} \sigma^{(\rho)}$ has to be computed to update $\sigma^{(\mu)}$ register. If $x^{(\mu-\rho)} \sigma^{(\rho)}(x)$ is stored in the $\sigma^{(\rho)}$ register instead of $\sigma^{(\rho)}(x)$, computationally intensive barrel shifters can be avoided. An effective way to achieve this is to "shift by 1" or multiplying by x before loading into $\sigma^{(\rho)}$ register. The initial value of $\sigma^{(\rho)}$ register must be x instead of 1. "load sigma" is used to update $\sigma^{(\rho)}$ register. After $2t$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

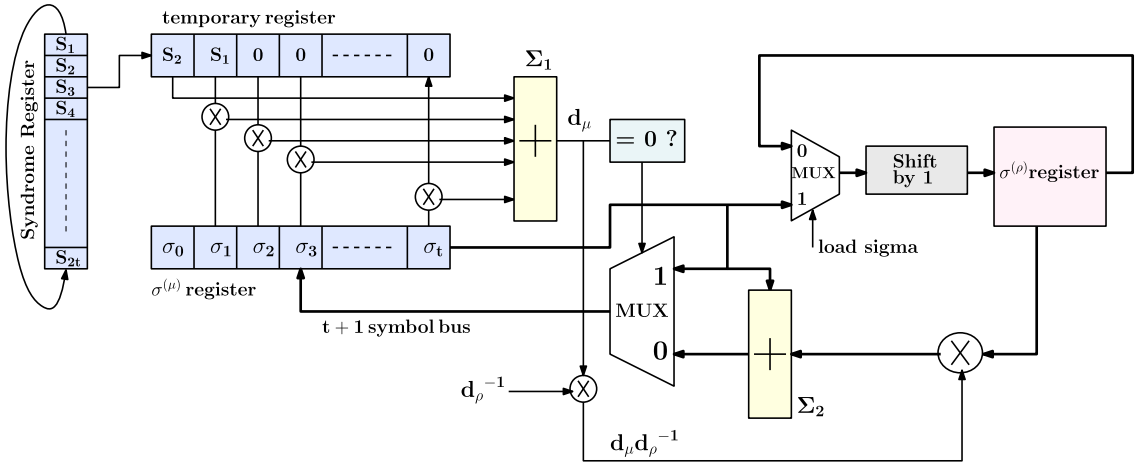


Fig. 10. Architecture of BM engine. The main steps in this stage are to find the discrepancy factor (d_μ) and the correction factor $d_\mu d_\rho^{-1} x^{(\mu-\rho)} \sigma^{(\rho)}(x)$, which will be used to update $\sigma^{(\mu+1)}(x)$ for the next iteration [1].

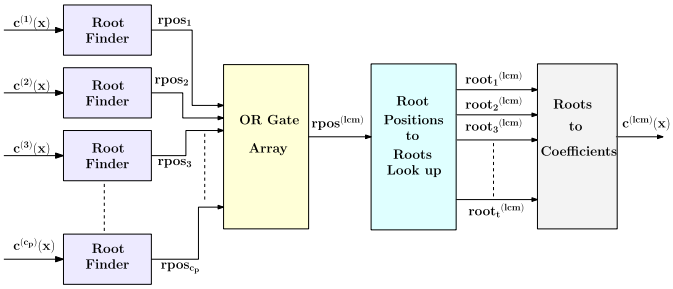


Fig. 11. Architecture of the common connection polynomial finder. Root finders find root positions of the connection polynomials, followed by an OR array which finds positions of common roots. The roots are then converted to the coefficients of the common connection polynomial.

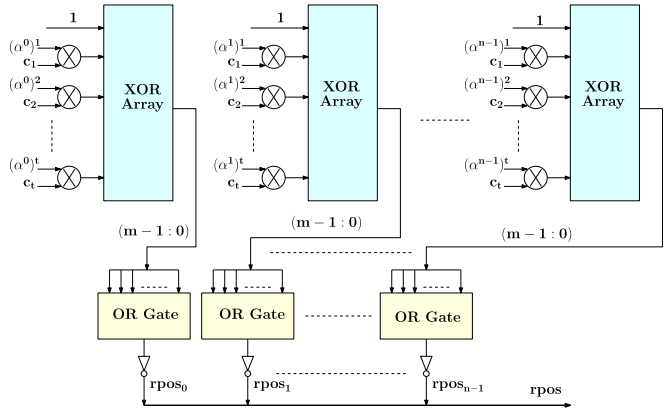


Fig. 12. Architecture of the root finder unit. n parallel stages test whether each element in GF(2^m) is a root of the connection polynomial.

iterations, $\sigma^{(\mu)}$ register contains the connection polynomial for a row (column).

3) Common Connection Polynomial Finder: Fig. 11 shows a design architecture, which finds the common connection polynomial between non-conjugate rows (columns) $1, 2, \ldots, c_p$. Reference [15, Remark 7] states that if $c_m(x)$ and $c_n(x)$ are connection polynomials of two rows (columns) m and n, then the lcm of $c_m(x)$ and $c_n(x)$ can be used to perform RE over both rows (columns) m and n. Thus, the lcm of the connection polynomials of the rows (columns) is the common row (column) connection polynomial. We use the following technique to find the lcm of the connection polynomials. Roots of the connection polynomials for non-conjugate rows (columns) are found using the architecture in Fig. 12, similar to the architecture used in [13].

The root finder consists of n parallel stages as shown in Fig. 12, which test whether each element of GF(2^m) is a root of $c(x) = c_0 + c_1 x + \cdots + c_t x^t$. This is done by using a GF(2^m) multiplier to multiply $(\alpha^i)^j$ with c_j for $0 \le i \le n-1$ and $1 \le j \le t$ and XOR-ing the results. If the output of the NOR gate for a particular field element is 1, it implies that it is a root. The root finder returns position of the roots, i.e., it returns an n-bit vector rpos, where bit 0, bit 1, …, bit $n-1$ represent whether $\alpha^0, \alpha^1, \ldots, \alpha^{n-1}$ are roots. 1 implies that a particular field element is a root and 0 implies that it is not a root of the connection polynomial.

We know that the union of the roots of the input connection polynomials is the roots of the lcm of the connection polynomials, i.e., the roots of the common connection polynomial. This is achieved by passing $rpos_{1:c_p}$ through an OR gate array to find $rpos^{(lcm)}$, which represents the root positions of the common connection polynomial. The roots positions are converted into the roots of the common connection polynomial using a lookup table. Let the roots of the lcm be $a_1, a_2, \ldots, a_{t_1}$. Thus, the common connection polynomial can be represented as

$$c^{(lcm)}(x) = (x + a_1)(x + a_2) \ldots (x + a_{t_1}). \quad (18)$$

The right-hand side of (18) can be expanded to get

$$\begin{aligned} c^{(lcm)}(x) &= (x + a_1)(x + a_2) \ldots (x + a_{t_1}) \\ &= 1 + c_1^{(lcm)} x + c_2^{(lcm)} x^2 + \cdots + c_{t_1}^{(lcm)} x^{t_1}. \end{aligned} \quad (19)$$

It can be easily seen in (19) that $\{c_p^{((lcm)}\}_{p=1}^{t_1}$ can be represented as a combination of $\{a_p\}_{p=1}^{t_1}$. We give a simple

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

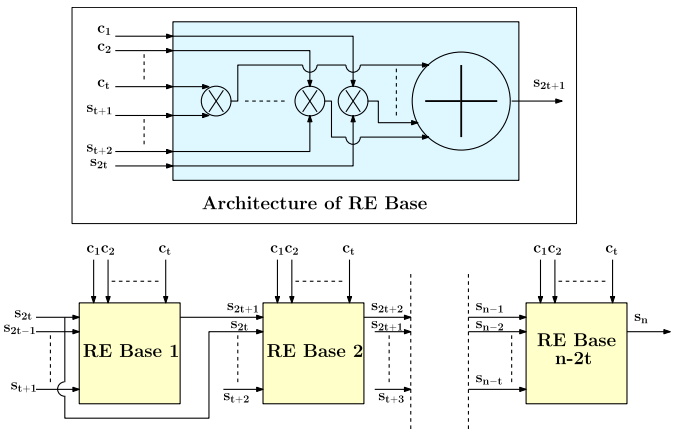


Fig. 13. Hardware architecture of the RE unit. RE base consists of GF multipliers and a XOR array, which produces the next syndrome S_{2t+1}. Cascaded stages of the RE base unit can be used to produce the syndromes of the entire row/column in a single clock cycle.

example to show this. Let $t_1 = 2$, giving us

$$\begin{aligned} c^{(\text{lcm})}(x) &= (x + a_1)(x + a_2) \\ &= a_1 a_2 + (a_1 + a_2)x + x^2 \qquad (20) \\ &= 1 + \frac{a_1 + a_2}{a_1 a_2} x + \frac{1}{a_1 a_2} x^2 \\ &= 1 + c_1^{(\text{lcm})} x + c_2^{(\text{lcm})} x^2. \qquad (21) \end{aligned}$$

Thus, $c_1^{(\text{lcm})} = ((a_1 + a_2)/(a_1 a_2))$ and $c_2^{(\text{lcm})} = (1/(a_1 a_2))$. This is realized by the "Roots to Coefficients" block in Fig. 11, the output of which is the required common connection polynomial.

4) RE Unit: The architecture of the RE unit is shown in Fig. 13. It consists of a base RE unit that evaluates (14). It consists of GF(2^m) multipliers that perform the multiplications and a XOR array that performs additions of all the multiplier outputs. The RE along a row/column can be carried in a serial fashion using the same RE base. However, this would require an $n - 2t$ number of clock cycles. To avoid this, we propose an architecture to perform the RE in a cascade fashion as shown in Fig. 13, where $n - 2t$ number of RE base units are used to perform the RE along a complete row/column in a *single clock* cycle. Needless to say, this will lead to a large propagational delay. Thus, depending on the throughput requirement, the RE along a row/column can be divided into stages. For example, if we divide it into two stages, two clock cycles will be required to perform the RE along a row/column. Thus, the latency will increase, but the delay of the RE along a row/column will be approximately halved.

5) 2-D IDFFFT Unit: The architecture of the 2-D IDFFFT unit is shown in Fig. 14(a). It consists of n^2 parallel IDFFFT units that perform 2-D IDFFFT for all (i, i'), $0 \le i, i' \le n-1$ in a single clock cycle. Each IDFFFT unit shown in Fig. 14(b) consists of cc_e parallel conjugate class units, where cc_e is equal to the total number of conjugate classes in $[C_{j,j'}]$. Each conjugate class unit has two inputs $(\alpha^{-(ij+i'j')})^{\text{IP}}_{\text{cc}_l}$ and $S^{\text{IP}}_{\text{cc}_l}$ corresponding to $[\hat{C}_{j,j'}]$ and $[C_{j,j'}]$, respectively. The conjugate class unit performs a combinational logic operation as described in Section III-B, thus achieving significant hardware complexity reduction similar to the IDFFFT units

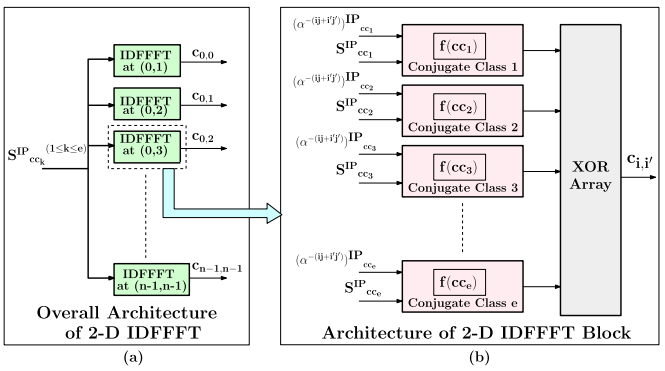


Fig. 14. (a) Hardware architecture of the 2-D IDFFFT unit. It consists of n^2 parallel IDFFFT units that produce the IDFFFT at each point in the $n \times n$ array. (b) Each IDFFFT unit consists of parallel conjugate class units to evaluate the IDFFFT at a certain point.

in the proposed encoder architecture in Section III-B. Finally, an XOR array is used to XOR all the outputs of the conjugate class units to produce the IDFFFT at a certain point (i, i').

F. Pipelining of the Decoder

The decoder consists of six stages. Other than the BM engine which takes $2t$ clock cycles, all other stages take 1 clock cycle each. The complete architecture can be pipelined as shown in Fig. 15 to achieve a latency of $(2t + 5)$ clock cycles. Each codeword takes $(2t + 5)$ clock cycles to be processed. After the first codeword has been processed, we get an output codeword every $2t$ clock cycles.

V. Design Implementation Results

Encoder and decoder hardware architectures for a 2-D BCH code of dimensions 15×15 with $t = 2$ were designed as a prototype example. The reader must note that the architecture is scalable for various block lengths and error correction ability. The designed 2-D BCH decoder can correct cyclic and quasi-cyclic burst of size 2×2. The architecture was implemented using XILINX ISE on a Kintex-7 KC-705 FPGA kit at 100 MHz. However, the post-place-and-route (PAR) reports a max frequency of 709.72 and 111.36 MHz for the encoder and decoder, respectively. The resource utilization details from the post PAR reports are summarized in Table IV.

Various parameters, such as the number of LUTs, registers, maximum frequency, and maximum throughput, are reported. Power numbers were generated from the XPower Analysis tool in Xilinx ISE. The encoder produces output in one clock cycle. The decoder is a pipelined architecture having six stages, consuming 1, 4, 1, 1, 1, and 1 clock cycles, respectively. The design has a latency of nine clock cycles, and after the first code is processed, we get an output every four clock cycles.

The throughput can be calculated as

$$\begin{aligned} \text{Throughput}_{\text{decoder}} &= \frac{n^2 \times f_{\text{dec}}}{2t} = \frac{225 \times f_{\text{dec}}}{4} \\ \text{Throughput}_{\text{encoder}} &= n^2 \times f_{\text{enc}} = 225 \times f_{\text{enc}} \end{aligned}$$

where f_{enc} and f_{dec} are the frequencies of operation of encoder and decoder, respectively.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

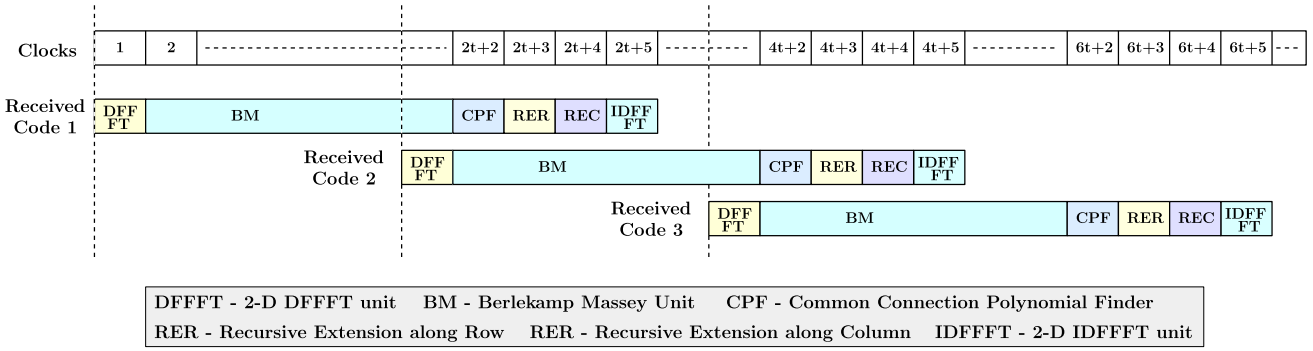


Fig. 15. Decoder consists of six pipelined stages, namely the 2-D DFFFT unit, the BM engine, the common connection polynomial finder, RE unit along row, RE unit along column, and the 2-D IDFFFT unit. They consume 1, $2t$, 1, 1, 1, and 1 clock cycles, respectively. The pipelined architecture has a latency of $2t + 5$ clock cycles and produces an output every $2t$ clock cycles after the first codeword is processed.

TABLE IV
KINTEX-7 KC-705 RESOURCE UTILIZATION SUMMARY

Parameters	Proposed 2-D BCH Encoder	Proposed 2-D BCH Decoder	Product Code Encoder [13]	Product Code Decoder [13]
Rate	181/225	181/225	49/225	49/225
LUTs	720	4993	523	2294
Registers	225	1082	436	1536
Min Period	1.409 ns	8.98 ns	1.416 ns	3.199 ns
Max. Frequency	709.72 MHz	111.36 MHz	706.43 MHz	312.6 MHz
Max. Throughput	159.69 Gbps	6.26 Gbps	158.9 Gbps	17.583 Gbps
Operating throughput (100 MHz)	22.5 Gbps	5.6 Gbps	22.5 Gbps	5.6 Gbps
Latency (in clock cycles)	1	9	1	6
Power (XPower Analyser)	0.255 W	0.432 W	-	-

We also compare the proposed encoder and decoder with the design of a product of two $(15, 7)$ BCH codes which we had designed earlier [13].[5] The product code has a low rate of 49/225 due to the construction and thus not suitable for correcting square bursts of large sizes. In addition, the product code decoder is able to correct an $n \times t$ burst only if it is known whether the burst has happened along the row or column. Without this information, the product code decoder would have twice the hardware requirement to process rows and columns parallely and one of those would result in the correct output. Due to the above reasons, the proposed 2-D BCH decoder is more suitable for applications in TDMR due to its flexibility in terms of code rate and burst shapes/sizes.

In Fig. 16, we show the error correction in a 2-D BCH code of size 15×15 with a quasi-cyclic burst error correction capability 2×2. The image is obtained by replacing every "1" and "0" with a white and black square in a 2-D BCH code. The red box indicates the 2×2 burst error that was successfully corrected using the designed hardware.

[5]We are not aware of any other 2-D BCH design architecture in the literature relevant for comparisons with the design architecture proposed in this article.



Fig. 16. Error correction in a 15×15, $t = 2$ 2-D BCH code. The red box shows a 2×2 burst that was successfully corrected using the designed hardware.

VI. CONCLUSION

We designed hardware architectures of an encoder and decoder for an $n \times n$ 2-D BCH code with quasi-cyclic burst error correction capability $t \times t$, having latencies of 1 and $2t + 5$ clock cycles, respectively. An encoder and decoder for a particular case of $n = 15$ and $t = 2$, and latencies of one and nine clock cycles, respectively, were implemented and tested targeting a Kintex 7 KC-705 FPGA kit. The proposed system architecture is easily scalable for different code lengths and ECC ability. The proposed 2-D BCH encoder and decoder are more suitable than BCH product codes for applications where flexibility of code rates and burst sizes/shapes is required.

We discussed stacking of 2-D BCH codes in 3-D for correction of $t \times t \times n$ bursts. This naturally motivates us toward the study of native 3-D spectra and 3-D encoding and decoding mechanisms for correcting arbitrary burst shapes in 3-D for applications in emerging 3-D storage applications.

ACKNOWLEDGMENT

This work was supported by the Ministry of Electronics and Information Technology, Govt. of India under Grant MITO0101.

REFERENCES

[1] A. Mondal, S. Thatimattala, V. K. Yalamaddi, and S. S. Garani, "Efficient coding architectures for Reed–Solomon and low-density parity-check decoders for magnetic and other data storage systems," *IEEE Trans. Magn.*, vol. 54, no. 2, pp. 1–15, Feb. 2018.

[2] M. Bahrami, C. K. Matcha, S. M. Khatami, S. Roy, S. G. Srinivasa, and B. Vasic, "Investigation into harmful patterns over multitrack shingled magnetic detection using the Voronoi model," *IEEE Trans. Magn.*, vol. 51, no. 12, pp. 1–7, Jul. 2015.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.

[3] R. Wood, R. Galbraith, and J. Coker, "2-D magnetic recording: Progress and evolution," *IEEE Trans. Magn.*, vol. 51, no. 4, pp. 1–7, Apr. 2015.
[4] Y. Kim *et al.*, "Three-dimensional NAND flash architecture design based on single-crystalline stacked array," *IEEE Trans. Electron Devices*, vol. 59, no. 1, pp. 35–45, Jan. 2012.
[5] S. G. Srinivasa, O. Momtahan, A. Karbaschi, S. W. McLaughlin, F. Fekri, and A. Adibi, "Volumetric storage limits and space-volume multiplexing trade-offs for holographic channels," *J. Opt. Eng.*, vol. 49, no. 1, Jan. 2010, Art. no. 015201.
[6] C. K. Matcha, S. Roy, M. Bahrami, B. Vasic, and S. G. Srinivasa, "2-D LDPC codes and joint detection and decoding for two-dimensional magnetic recording," *IEEE Trans. Magn.*, vol. 54, no. 2, pp. 1–11, Feb. 2018.
[7] S. S. Garani, L. Dolecek, J. Barry, F. Sala, and B. Vasic, "Signal processing and coding techniques for 2-D magnetic recording: An overview," *Proc. IEEE*, vol. 106, no. 2, pp. 286–318, Feb. 2018.
[8] C. K. Matcha, M. Bahrami, S. Roy, S. G. Srinivasa, and B. Vasic, "Generalized belief propagation based TDMR detector and decoder," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Barcelona, Spain, Jul. 2016, pp. 210–214.
[9] H. Imai, "A theory of two-dimensional cyclic codes," *Inf. Control*, vol. 34, no. 1, pp. 1–21, May 1977.
[10] R. E. Blahut, "Transform techniques for error control codes," *IBM J. Res. Develop.*, vol. 23, no. 3, pp. 299–315, May 1979.
[11] S. Roy and S. G. Srinivasa, "Two dimensional error-correcting codes using finite field Fourier transform," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Oct. 2015, pp. 119–123.
[12] P. Elias, "Error-free coding," *Trans. IRE Prof. Group Inf. Theory*, vol. 4, no. 4, pp. 29–37, Sep. 1954.
[13] A. Mondal and S. S. Garani, "Efficient hardware design architectures for BCH product codes in the frequency domain," in *Proc. IEEE 63rd Int. Midwest Symp. Circuits Syst. (MWSCAS)*, Springfield, MA, USA, Aug. 2020, pp. 703–706.
[14] A. Gunasekaran, N. C. Jose, and S. S. Garani, "An optimized BCH decoder design architecture for adaptive M-ary recording channels," in *Proc. IEEE 62nd Int. Midwest Symp. Circuits Syst. (MWSCAS)*, Dallas, TX, USA, Aug. 2019, pp. 710–713.
[15] H. S. Madhusudhana and M. U. Siddiqi, "On Blahut's decoding algorithms for two-dimensional BCH codes," *IEEE Trans. Inf. Theory*, vol. 44, no. 1, pp. 358–367, Jan. 1998.
[16] H. Okano and H. Imai, "A construction method of high-speed decoders using ROM's for Bose–Chaudhuri–Hocquenghem and Reed–Solomon codes," *IEEE Trans. Comput.*, vol. C-36, no. 10, pp. 1165–1171, Oct. 1987.
[17] S. G. Srinivasa, P. Lee, and S. W. McLaughlin, "Post-error correcting code modeling of burst channels using hidden Markov models with applications to magnetic recording," *IEEE Trans. Magn.*, vol. 43, no. 2, pp. 572–579, Feb. 2007.
[18] C. K. Matcha and S. G. Srinivasa, "Generalized partial response equalization and data-dependent noise predictive signal detection over media models for TDMR," *IEEE Trans. Magn.*, vol. 51, no. 10, pp. 1–15, Oct. 2015.
[19] S. Lin and D. J. Costello, *Error Control Coding*, 2nd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2014.
[20] X. Zhang, *VLSI Architectures for Modern Error-Correcting Codes*. Boca Raton, FL, USA: CRC Press, 2015.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on August 15,2026 at 07:53:59 UTC from IEEE Xplore. Restrictions apply.